

Desenvolvimento com IA — Como Trabalhamos, Riscos e Cuidados

Leia este documento antes de qualquer outro. Ele explica por que nosso fluxo de desenvolvimento com IA é estruturado do jeito que é.

1. Por que este documento existe

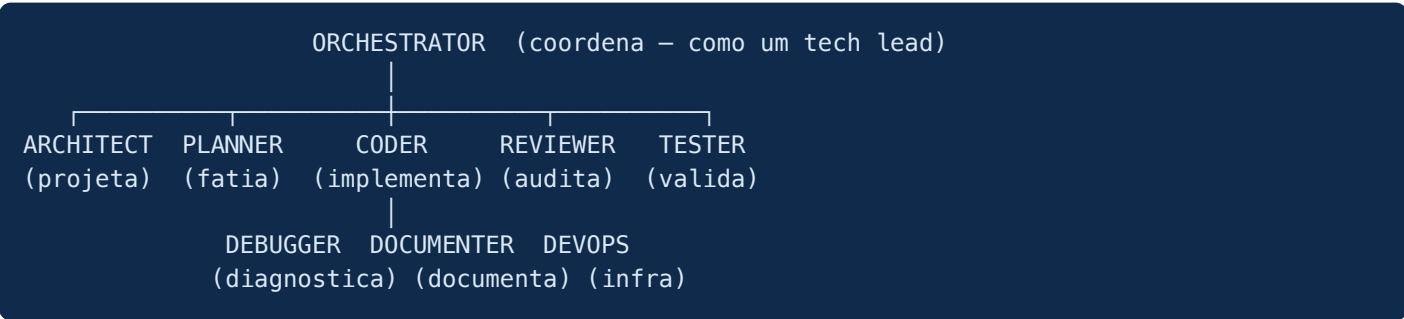
IA acelera desenvolvimento de forma real — mas **sabemos que implementar com IA tem riscos e exige cuidados**. Sem método, os problemas aparecem rápido:

- Código gerado que **parece certo e está errado** (APIs alucinadas, edge cases ignorados, lógica sutilmente quebrada).
- Mudanças grandes demais para alguém revisar de verdade — o review vira teatro.
- Ações destrutivas executadas com confiança: push em `main`, `rm -rf`, reset de histórico.
- Segredos e credenciais parando em código ou em contexto de IA.
- Perda de rastreabilidade: ninguém sabe o que foi decidido, por quem, nem por quê.

Nossa postura é simples: **IA com processo de engenharia, não IA no improviso**. Cada risco acima tem uma mitigação concreta neste playbook — por método e por trava mecânica, não por confiança.

2. Como funciona o time de agentes

Não usamos "uma IA que faz tudo". Usamos um **time de agentes especializados**, espelhando um time real de desenvolvimento:



Agente	Papel no time	Equivalente humano
Orchestrator	Decompõe o pedido, delega, coordena, sintetiza. Não escreve código.	Tech Lead
Architect	Design de sistema e decisões técnicas, antes de qualquer código	Arquiteto de Software
Planner	Quebra o design em tasks pequenas com critérios de aceitação	Analista / PO técnico
Coder	Implementa exatamente o que a spec pede	Desenvolvedor
Reviewer	Audita segurança, performance e correção — gate obrigatório	Revisor sênior
Tester	Escreve e executa testes; valida critérios de aceitação	QA
Debugger	Reproduz, isola e diagnostica root cause	Especialista em troubleshooting
Documenter	READMEs, API docs, runbooks — só após aprovação	Technical Writer
DevOps	CI/CD, deploy, infra — com checklist pré-deploy	Engenheiro DevOps

Por que separar papéis reduz erro:

- 1. **Especialização** — cada agente faz uma coisa e faz bem; não mistura responsabilidades.
- 2. **Revisão cruzada** — quem escreve o código nunca é quem aprova. O Coder não revisa o próprio trabalho, exatamente como num time maduro.
- 3. **Escopo limitado por ferramenta** — cada agente só tem acesso às ferramentas do seu papel. O Reviewer não edita código (só lê); o Planner não executa comandos; o Debugger não corrige em produção (entrega diagnóstico ao Coder).
- 4. **Handoff estruturado** — o trabalho passa de agente para agente via artifacts gravados em disco, com rastro auditável (`tasks/{id}/run-log.md`). Nada se perde em "telefone sem fio".

Detalhe completo: [multi-agents/ARCHITECTURE.md](#) e [multi-agents/HANDOFF-PROTOCOL.md](#).

3. O modelo certo para cada função

Modelos de IA têm capacidades e custos diferentes. Concentramos o modelo mais capaz **onde o erro custa caro** — decisão, design e o gate de qualidade — e usamos modelos eficientes na execução, que já trabalha sob spec fechada e passa pelo Reviewer de qualquer forma.

Agente	Modelo	Por quê
Orchestrator	Fable 5	A decomposição define a qualidade de tudo que vem depois
Architect	Fable 5	Decisões de arquitetura são caras de reverter
Reviewer	Fable 5	É o gate: um falso "aprovado" é o erro mais caro do fluxo
Planner	Sonnet 5	Estrutura trabalho sobre design já decidido
Coder	Sonnet 5	Implementa spec fechada; erro é pego pelo Reviewer
Tester	Sonnet 5	Método estruturado, critérios já definidos
Debugger	Sonnet 5	Segue metodologia científica de debugging
DevOps	Sonnet 5	Procedural, com checklist
Documenter	Haiku 4.5	Alto volume, insumo já aprovado

Racional completo e alternativas descartadas: [docs/ADR-001-modelos-por-agente.md](#).

4. Os riscos que reconhecemos — e como cada um é mitigado

Risco real de dev com IA	Nossa mitigação
Código errado ou alucinado que "parece certo"	Reviewer obrigatório (Fable 5): nenhuma task fecha sem <code>artifacts/reviewer.md</code> com aprovação explícita — verificado por hook, não por boa vontade
Ação destrutiva no repositório	Hooks bloqueiam push em <code>main / master (inclusive HEAD:main)</code> , force push, <code>rm -rf</code> , <code>git reset --hard</code>
Mudança grande demais para revisar	Planner fatia em mudanças de ~200–400 linhas; diff maior que isso é sinalizado no review
Escopo descontrolado (IA "aproveita para arrumar" o que ninguém pediu)	Delegação com LIMITES explícitos ; um dono por arquivo; agente fora do escopo = anti-padrão registrado
Contexto se perdendo entre etapas	Artifacts completos em disco + ponteiros leves; <code>run-log.md</code> append-only com a linha do tempo de cada task
Segredo/credencial em código	Regra dura no Coder (sem hardcoded) + item obrigatório do checklist do Reviewer; segredos só em cofre/secrets do CI
Excesso de confiança ("a IA disse que está pronto")	Nada é <code>done</code> sem passar pelo gate; o dev revisa o PR antes de marcá-lo ready — responsabilidade final é humana
A própria IA alterar suas travas	<code>settings.json</code> , hooks e GOVERNANCE são zonas negadas para escrita pelos agentes

5. As travas mecânicas (não são opcionais)

Regra de processo depende de obediência; **trava mecânica não**. O kit traz as duas camadas:

- `.claude/settings.json` → `permissions.deny` — nega de saída: push direto em `main`, force push, `rm -rf`, `git reset --hard`, `git clean -fd`, e qualquer escrita em `GOVERNANCE.md`, `.claude/settings.json` e `.claude/hooks/`.
- **Hooks (scripts que interceptam as ações da IA):**
 - `block-dangerous-git.sh` — analisa cada comando antes de executar; bloqueia variações que burlam o deny simples (ex.: `git push origin HEAD:main`, flags reordenadas).
 - `protect-guardrails.sh` — impede a IA de editar as próprias travas e a governança.
 - `check-reviewer-gate.sh` — no fim de cada sessão, verifica se alguma task foi marcada `done` sem aprovação do Reviewer; se sim, bloqueia o encerramento.

Regra de ouro: trava disparou → **não se contorna**. Nem manualmente "só dessa vez". Se parecer falso positivo, reporte ao Tech Lead — a trava é ajustada pelo processo, nunca ignorada.

6. Boas práticas de desenvolvimento com IA

1. **Branch + draft PR sempre.** `main` é intocável. Todo trabalho nasce em branch, sobe como draft PR e só vira ready depois de revisado.
 2. **Mudanças pequenas.** ~200–400 linhas por PR. Se cresceu, volta pro Planner fatiar.
 3. **Critérios de aceitação antes do código.** Se você não sabe dizer quando está pronto, não está pronto para começar.
 4. **Testes são contrato, não enfeite.** O Tester valida os critérios; teste falhando → Debugger, não gambiarra.
 5. **Você é responsável pelo que a IA produziu em seu nome.** Leia o diff inteiro antes de abrir o PR. "Foi a IA" não é justificativa em code review.
 6. **Commits limpos.** Mensagens descritivas do quê e por quê. Sem marcações de IA no código ou nos commits.
 7. **Um dono por arquivo por vez.** Trabalho paralelo usa git worktrees e arquivos disjuntos (ver `GOVERNANCE.md` §4).
 8. **Registre decisões.** Decisão técnica relevante vira ADR em `docs/` — quem chegar depois entende o porquê.
 9. **Na dúvida, pergunte.** Ambiguidade que muda a abordagem → pergunta ao Tech Lead antes, não suposição depois.
-

7. O que a IA NUNCA faz aqui

- Merge ou push em `main/master`.

- Deploy sem o checklist pré-deploy do DevOps.
- Criar, ler ou manipular credenciais e segredos.
- Apagar dados ou histórico (`rm -rf` , force push, hard reset).
- Alterar as próprias travas: `.claude/settings.json` , `.claude/hooks/` , `GOVERNANCE.md` .
- Marcar trabalho como concluído sem passar pelo gate do Reviewer.

8. Onde aprender o resto

Documento	O que cobre
ONBOARDING.md	Passo a passo para começar a usar o fluxo
GOVERNANCE.md	Papéis, regras de git, gates e travas — a "lei" do playbook
multi-agents/ARCHITECTURE.md	Doutrina: topologia, fluxos, anti-padrões, scaling de esforço
multi-agents/HANDOFF-PROTOCOL.md	Como os agentes trocam trabalho na prática
multi-agents/SKILLS-GOVERNANCE.md	Quando reutilizar/criar skills (regra dos 3)
docs/ADR-001-modelos-por-agente.md	Por que cada agente usa o modelo que usa
multi-agents/agents/NN-*.md	Manual completo de cada agente